Week 1 · Day 4: Building with LLMs

AI Intern Onboarding · Daily Task Sheet · Thursday, 8 October 2026

Today's goal: move from understanding LLMs to building with them. Call a model through an API, write prompts that give reliable answers, get structured JSON back and validate it, and understand when RAG or fine-tuning is the better tool. Today you write the first piece of the Note Summariser: a Python function that sends a sample visit note to the model and returns a validated JSON summary (main complaint, medications, follow-up).

Data rule: use only public, de-identified or synthetic data today (and every day). No real patient information (PHI) in any exercise, notebook, prompt or daily note. Every visit note you send to the model today must be made up by you. Anything you put in a prompt leaves your laptop and goes to a cloud service, which is one more reason PHI never goes in.

Schedule (shift 2:00 pm to 11:00 pm)

Time	Block	Activity
14:00–14:15	Setup	Create a *week1-day4* notebook or project folder. Install `openai`, `pydantic` and `python-dotenv`. Put your sandbox Azure OpenAI key and endpoint in a `.env` file and add it to `.gitignore`. Make one test call.
14:15–17:45	Study (3.5 h)	Topics 1–5 below, in order. Take short notes as you go; about 40 min per topic.
17:45–18:30	Meal break	
18:30–21:30	Hands-on (3 h)	Project task: build the first version of the Note Summariser function (steps below).
21:30–22:30	Discussion (1 h)	Explain today's ideas to your peers and mentor (prompts below).
22:30–23:00	Daily note (0.5 h)	One page in your own words (template below).

Study topics

1. Calling an LLM through an API: messages, tokens and cost

- **What:** you send the model a list of *messages* and it sends back the next message. The *system* message sets the rules and role ("You summarise visit notes. Only use facts in the note."); *user* messages carry the request and the input; *assistant* messages are the model's replies. Settings such as temperature and max tokens travel with the request.
- **Azure OpenAI:** the same OpenAI models hosted in our Azure subscription. You call a *deployment* (a named copy of a model) at your resource's endpoint, using the OpenAI Python SDK.
- **Tokens and cost:** you pay for input tokens (system + user messages) and output tokens, and output tokens usually cost more. The response tells you how many were used. Long system prompts are paid for on every single call.
- **Where it breaks:** APIs fail sometimes (rate limits, timeouts, network errors), so production code needs retries with back-off and a clear error path. Keys are secrets: never in code, notebooks or git.
- **Do:** estimate the cost of summarising 1,000 notes of about 400 tokens each, with a 150-token summary, using current prices from the Azure pricing page.

2. Prompt engineering: clear instructions, examples, output format

- **What:** the prompt is your spec. Good prompts say who the model is, what the task is, what to do when information is missing, and exactly what shape the answer must take.
- **Techniques:** be specific ("list each medication with dose if stated" beats "summarise"); separate instructions from input with clear delimiters (e.g. tags around the note); give one or two worked examples (*few-shot*); tell the model what to output when a field is absent (e.g. an empty list, not a guess).
- **Why it matters:** most quality problems you will see tomorrow are fixed by a clearer prompt, not a bigger model.
- **Where it breaks:** prompts are fragile: small wording changes can change answers, so every prompt change needs re-testing against your examples (Day 5).
- **Do:** rewrite the prompt "Summarise this note" into a version a careful new colleague could follow without asking questions.

3. Structured output: asking for JSON and validating it

- **What:** code cannot reliably use free text, so you ask for JSON with fixed fields. JSON mode makes the model return valid JSON; *structured outputs* go further and make it follow a schema you give it.
- **Pydantic:** a Python library where you describe the shape once as a class (e.g. `main_complaint: str`, `medications: list[Medication]`, `follow_up: str | None`) and it checks and parses the data, raising a clear error if anything is wrong. The same class can produce the JSON schema you send to the model.
- **Why it matters:** valid JSON is not the same as correct content. Validation catches wrong shapes and types; it does not catch a medication the model invented. That needs tests (Day 5) and a clinician.
- **Do:** write the Pydantic model for the summary on paper, including which fields may be empty.

4. Retrieval-augmented generation (RAG), at the idea level

- **What:** instead of hoping the model remembers a fact, you find the relevant documents first and put them in the prompt, then ask the model to answer only from them. Steps: split documents into chunks, turn chunks into embeddings (Day 2), find the chunks closest to the question, and pass them to the model with the question.
- **Why it matters:** answers can use private or up-to-date documents the model was never trained on, can cite their sources, and are easier to check. It reduces, but does not remove, hallucination.
- **Where it breaks:** if retrieval finds the wrong chunk, the model answers confidently from the wrong text. Chunk size, search quality and the context window all matter.
- **Do:** sketch the RAG flow for a tool that answers staff questions from our internal policy documents.

5. Prompting vs RAG vs fine-tuning: when to use which

- **Prompting:** always try first. Cheapest and fastest to change. Enough when the model already has the skill and the input fits in the prompt (our summariser).
- **RAG:** when the model needs *knowledge* it does not have: private, large or frequently changing documents, or when answers must point to a source.
- **Fine-tuning:** further training on many examples, when you need a consistent *behaviour* or style that prompting cannot reliably produce, or a smaller, cheaper model for one narrow task. Costs data, time and money, and the knowledge it adds goes stale.
- **Do:** for each of these, choose one: (a) answer from this week's updated formulary; (b) always write summaries in our house format; (c) summarise one note into JSON.

Hands-on project: Note Summariser v0.1

Goal: a Python function `summarise_note(note: str) -> NoteSummary` that sends a made-up visit note to Azure OpenAI and returns a validated summary with the main complaint, medications and follow-up. **Tools:** the `openai` package (its Azure client), `pydantic` and your sandbox Azure OpenAI deployment from your mentor. **Notes:** write 3 short visit notes yourself: one simple, one with several medications, and one with no follow-up mentioned. No names, dates of birth, record numbers or details from real notes.

#	Step	Time
1	Load the key, endpoint and deployment name from `.env`. Make a first chat call with a system and a user message. Print the reply and the token usage from the response.	15 min
2	Define Pydantic models: `Medication` (name, dose, frequency, each optional except name) and `NoteSummary` (main_complaint, medications, follow_up). Validate a hand-written example and a broken one to see the error.	25 min
3	Write the system prompt: role, task, "use only facts stated in the note", what to return for missing information, and the exact JSON shape. Put the note inside clear delimiters in the user message.	25 min
4	Request JSON (JSON mode or structured outputs with your schema) at a low temperature. Parse the reply into `NoteSummary` with `model_validate_json`.	25 min
5	Wrap it all in `summarise_note()`. On a validation error, retry once with the error message added; then raise a clear exception. Catch API errors and timeouts.	30 min
6	Add one worked example (few-shot) to the prompt. Run all 3 notes with and without it and compare the results in a markdown cell.	25 min
7	Log the input and output tokens and estimated cost per call. Print a small table for your 3 notes.	15 min
8	Read every output against its note. Mark anything missing or invented. Tidy the code so it reruns cleanly and commit it (without the .env file).	20 min

Stretch (if time): add a *confidence* or *evidence* field quoting the sentence each medication came from, and check the quote really appears in the note; compare a smaller and a larger deployment on cost and quality; stream the response and print it as it arrives.

Challenge project: Grounded RAG assistant with citations and an evaluation

Difficulty: hard. Start it after the core project task; expect roughly 8 to 10 extra hours, so work on it in spare slots across Week 1 and present it on Friday alongside your summariser. Ask your mentor early if you get stuck.

Scenario: staff want quick answers to questions about medicines and care guidance, with the source shown so they can check it. Build a small RAG assistant over public documents that answers only from what it retrieved, cites its sources, says "I don't know" when the documents do not cover the question, and is measured honestly.

Data: a small set of public documents only, e.g. 20 to 50 public drug labels from openFDA or DailyMed, or public patient-information pages from MedlinePlus. Questions are written by you. Never use real clinical records or internal documents without your mentor's approval.

Milestones

#	Milestone	What to deliver
1	Ingest and chunk	Load the documents, clean them and split them into chunks with metadata (source, section). Try two chunk sizes and keep both for milestone 6.

#	Milestone	What to deliver
2	Embed and index	Create embeddings with an Azure OpenAI embedding deployment and store them in a simple index (NumPy, FAISS or Chroma). Write a search function that returns the top-k chunks with scores.
3	Grounded answers	Prompt the model to answer only from the retrieved chunks and to cite chunk IDs. Return a Pydantic object: answer, citations, and an "answerable" flag.
4	Citation check	In code, confirm every cited ID was actually retrieved and that quoted text appears in the chunk. Count and report citation errors.
5	Test set	Write 30 questions with known answers and their source passages, plus 10 that the documents cannot answer. Keep it in a JSON or CSV file.
6	Retrieval evaluation	Measure recall@k (is the right passage in the top k?) for k = 1, 3, 5 and both chunk sizes. Plot the results and choose settings.
7	Answer evaluation	Label answers correct, partly correct, wrong or correctly abstained; report rates with confidence intervals. Compare with the same model answering without retrieval.
8	Report	1–2 page markdown report: design, results, failure examples, cost per question, limitations, and what would be needed before staff could use it (review, monitoring, updates).

Success criteria

- Repo reruns end to end from a fresh environment (requirements file included; no keys in the code or history).
- Every answer has citations that pass the automatic check, or says it cannot answer.
- Retrieval and answer quality are reported with numbers, sample sizes and uncertainty.
- The report states plainly where the assistant should not be trusted.
- You can defend each design choice in a 10-minute walkthrough.

Stretch goals

- Add hybrid search (keyword plus embeddings) and measure whether recall improves.
- Add a re-ranking step on the top 20 chunks and compare recall@3.
- Try a query-rewriting step for vague questions and measure its effect.
- Build a tiny Streamlit or Gradio page that shows the answer next to the cited passages.

Resources to study from

Core (free)

Resource	Use it for
Microsoft Learn: Azure OpenAI documentation and quickstarts (learn.microsoft.com)	Creating a deployment, calling it from Python, JSON mode and structured outputs on Azure, and quotas.
OpenAI Python SDK on GitHub (github.com/openai/openai-python)	Installing the package, the Azure client, error types, retries and timeouts.
OpenAI platform docs: prompt engineering and structured outputs guides	Practical prompting techniques and how schema-based output works.
Pydantic documentation (docs.pydantic.dev)	Models, field types, optional fields, `model_validate_json` and generating JSON schema.

Resource	Use it for
DeepLearning.AI short courses, e.g. "ChatGPT Prompt Engineering for Developers" and a RAG course	Short, hands-on videos with notebooks on prompting and retrieval.
Microsoft Learn: Azure AI Search and RAG overview	How RAG is built on Azure; background for the challenge project.

Deeper reading

- Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' (2020): the paper that named RAG.
- Brown et al., 'Language Models are Few-Shot Learners' (GPT-3, 2020): why examples in the prompt work.
- Wei et al., 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models' (2022): asking for steps, and its limits.
- Gao et al., 'Retrieval-Augmented Generation for Large Language Models: A Survey' (2023): RAG designs and evaluation.
- Es et al., 'RAGAS: Automated Evaluation of Retrieval Augmented Generation' (2023): ideas for milestone 7.
- Chip Huyen, *AI Engineering* (O'Reilly, 2025): chapters on prompting, RAG, fine-tuning and evaluation.
- Anthropic and OpenAI prompt engineering guides: vendor-written tips that apply to most chat models.

Resource availability, prices and API details change over time; if a link or feature has moved, search for the title or ask your mentor.

Check yourself

When would you use RAG instead of fine-tuning?

Hint: ask whether the problem is missing *knowledge* or missing *behaviour*, how often the information changes, whether answers need to show a source, and how much labelled data and money you have. Use your answer to topic 5's exercise as evidence. Write your answer, then compare it with a peer's.

Discussion prompts (1 hour)

- Show your system prompt to a peer. Which instruction made the biggest difference, and how do you know?
- Your JSON passed Pydantic validation but listed a medication that is not in the note. What should catch that, and when?
- Where should the API key live in development, and where in production on Azure?
- Would RAG make the Note Summariser better? Why or why not?
- What surprised you in today's results?

Daily note template (one page)

- Three things I learned today (my own words).
- One thing that is still unclear, to ask my mentor.
- My answer to the check-yourself question.
- What I will do differently tomorrow.

Definition of done today: `summarise_note()` runs on your 3 made-up notes and returns validated summaries; errors and invalid JSON are handled; token use and cost are logged; outputs checked against the notes in writing; code committed without secrets; daily note submitted; questions logged for your mentor.

Looking ahead: Day 5 covers responsible AI and review: testing LLM output against expected answers, prompt injection, bias and fairness, and patient data. You will write 10 test notes for your summariser, check its answers, then try to break it with a prompt injection. In the afternoon you give a 10-minute talk to your mentor on what you built this week. Keep today's code tidy; tomorrow you test it.